Lab 2: Prompt Library and Department Assistant

AI Administrator · Day 1 · Session 2 · Prompting for Operators

Participant lab handout

Objective

Build two things you will reuse for the rest of the course: a five-template prompt library your team can copy, and a department assistant that answers only from uploaded documents, tested with a trap.

Time: 20 minutes in the session; finish at home. **Due:** before the next session.

What you need

- Any of ChatGPT, Claude, Microsoft Copilot or Gemini.
- A blank document for the library and a blank spreadsheet for the test set.
- Two **public** policy documents (for example a university's leave policy and a travel-expenses policy found online), or made-up ones. **Do not upload real internal documents yet.** Day 6 covers the data rules.

Steps

1. Create the library document

Open a new document called **M2-prompt-library**. Make five headings:

1. Triage an email
2. Summarise a meeting transcript
3. Draft a reply
4. Extract fields from a form
5. Classify a request

2. Write each template with the five parts

Under each heading write **Role, Context, Task, Constraints, Output format**. Use square-bracket placeholders such as `[department]` and `[paste text]` so a colleague can fill them in. Start from this skeleton:

```
Role: You are an assistant to the [role] in [department].
Context: [two or three facts and definitions the model must know]
Task: [one verb, one deliverable]
Constraints: Use only the information provided. If something is unclear or missing,
write "unclear". Maximum [N] words. Tone: [plain / formal].
Output format (exactly these lines):
[Field 1]:
[Field 2]:
[Field 3]:

Input: [paste text]
```

Keep the “unclear” sentence in every template. It is the most valuable line you will write today.

3. Add examples to the classifier

For **Classify a request**, list four allowed categories and write one example for each. Add a fifth, awkward example that should be classified as *other*. Pattern:

```
Classify each message as billing, technical, sales or other.
Example 1: "My invoice shows the old price" -> billing
Example 2: "The portal logs me out every minute" -> technical
Example 3: "Do you offer a plan for schools?" -> sales
Example 4: "Thanks for the quick help yesterday" -> other
Now classify: "[message]"
Answer with the category only.
```

4. Build the ten-row test set

Write ten realistic inputs for the classifier with the category you expect for each. Include two awkward ones (mixed intent, sarcasm, very short). Then use the course bot as a test bench: open **@aicaravan_bot** on Telegram and send one message in this shape:

```
<paste your classifier prompt here>

Test inputs with expected labels:
1) <input 1> | <expected>
2) <input 2> | <expected>
...
10) <input 10> | <expected>
```

The bot returns a table (input, expected, model answer, match) and a score. Copy the table into your library document: **score 1 = ___ / 10**. You can also run the same ten rows in your own assistant and compare.

Input	Expected	Model answer

5. Fix and re-run

For every mismatch, change the prompt by adding a **constraint or an example** that covers the failing case. Never add “please be more accurate”. Then paste the whole message to the bot again, **all ten rows**, and record **score 2 = ___ / 10**.

6. Create the department assistant

In your assistant, create a Project (ChatGPT), a Project (Claude), a Gem (Gemini) or a Copilot agent. Paste this as its instructions and upload your two public policy documents:

```
You are the [department] policy assistant. Answer questions only from the uploaded
documents.
For every answer, quote the sentence you relied on and name the document.
If the answer is not in the documents, reply: "Not covered by the uploaded policies.
Please ask [named person]." Keep answers under 120 words.
```

7. Test the assistant with a trap

Ask your assistant three questions the documents answer, then two they do not (for example a benefit that is not mentioned). Note whether it admits “not covered” or invents an answer.

Then run the same test on the course policy bot, **@aicaravan_policy_bot** on Telegram. It has the same instructions and two fictional AI Caravan Ltd. policies built in (leave; travel and expenses), no tools and no memory. Suggested questions:

Question	Expected
How many days of annual leave do I get?	Covered: 24 working days
How many unused days can I carry over?	Covered: 5, used before 31 March
What is the daily allowance inside Jordan?	Covered: 45 JOD
How many weeks of parental leave can I take?	Not covered
What is the mileage rate for using my own car?	Not covered

Record both sets of results at the bottom of your library document, and one line on where the two assistants differed.

Deliverable

The **M2-prompt-library** document containing: five templates, the classifier with examples, test scores before and after, and the assistant’s trap-test results; plus a screenshot of the assistant answering one question with a quotation. Bring it to the next session.

Stuck?

- **My product has no Projects / uploads.** Paste the policy text into the chat as context instead. Same idea, shorter memory. The course policy bot still works for the trap test.
- **The course bot does not answer.** The instructor switches it on for the session; ask them to check `hermes gateway status`.
- **The classifier scored 10/10 first time.** Add two more awkward inputs until something fails. The habit you are building is fix-and-re-run, not the score.
- **The assistant invented an answer to a trap question.** Good: you found it before a colleague did. Tighten the instruction ("Only if the exact wording appears in a document…") and test again.

For the instructor: quick rubric

Check	Pass looks like
Five templates	Each has all five parts and bracket placeholders; the "unclear" clause is present
Classifier	Four allowed values, four examples plus one awkward *other* example
Test set	Ten rows with expected and model answers; two scores recorded
Assistant	Instructions pasted, two public documents uploaded, screenshot shows a quotation
Trap test	Five results for their own assistant and five for the course policy bot; participant can say where the two differed

Reference: Module 2 online, with the knowledge check scored in the browser: albanna-tutorials.com/ai-administrator/02_prompting_for_operators.html